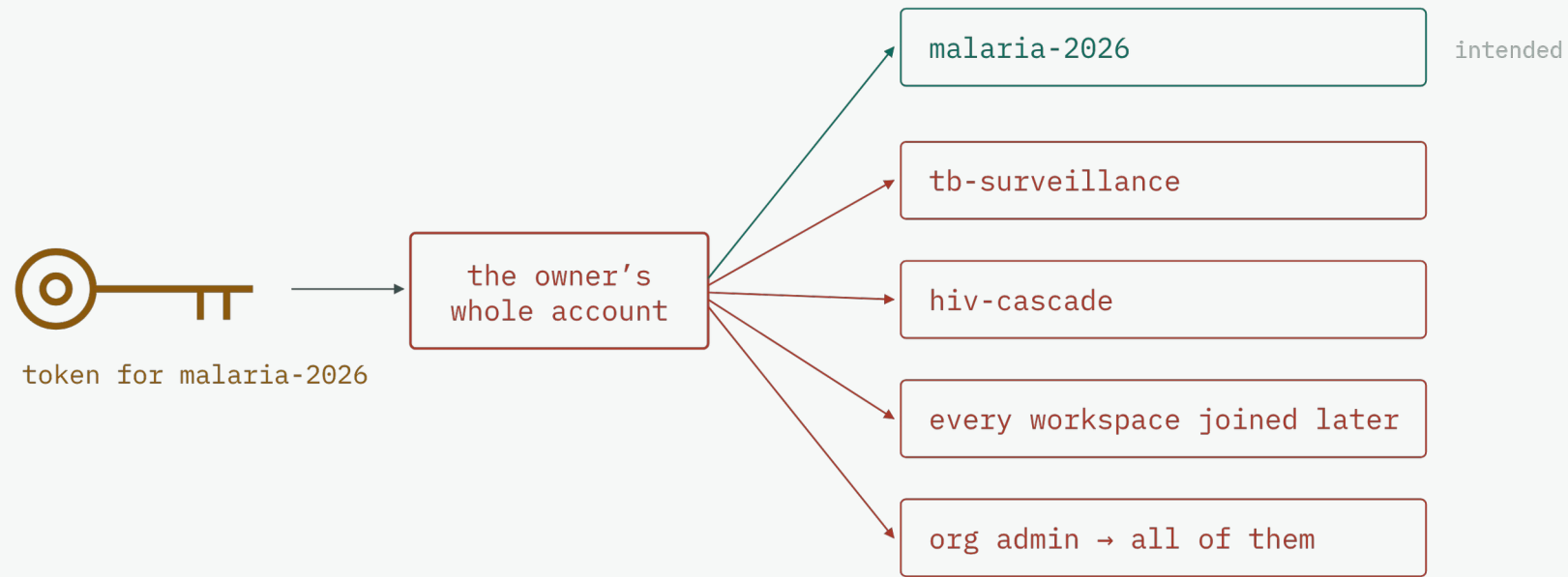


Workspace tokens that stop at the workspace

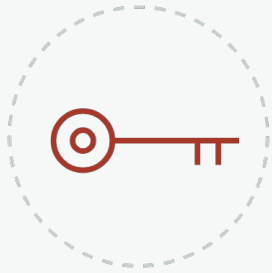
HEXA-1775 MIGRATION PLAN DRAFT — FOR DISCUSSION

A token is issued *for* a workspace. It isn't limited *to* one.

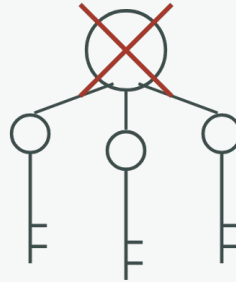


One key, every door its owner can open — today's doors and tomorrow's.

Three things we cannot do today



Limit it
the boundary isn't enforced



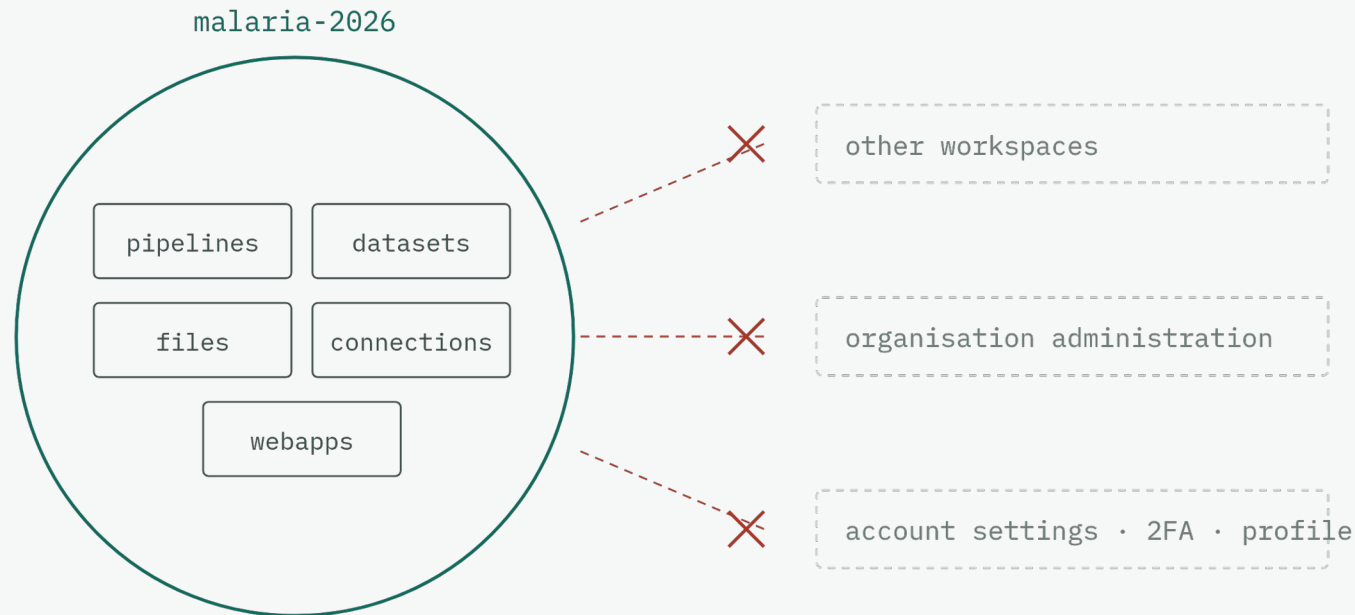
Revoke one
rotating breaks every copy



See usage
no name, no last-used, no owner

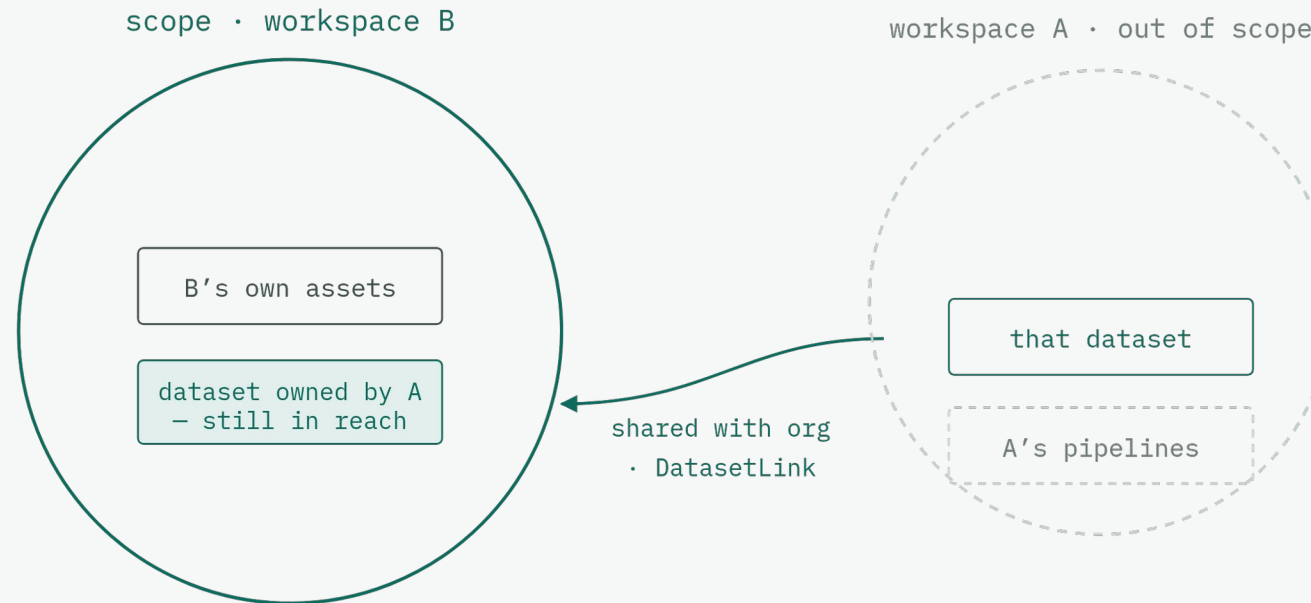
Also true today: a token bypasses 2FA, and its secret is stored in the database in plaintext.

Scope is a ring drawn around one workspace



A token acts as its owner, but only inside the ring. Superusers included — no wider than anyone else.

The ring follows *sharing*, not ownership



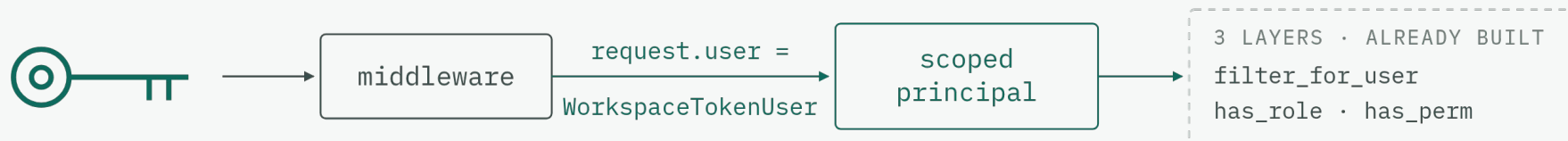
A token for B reaches what *B* reaches. Get this wrong and we break sharing, not just tokens.

Scoping is one line away

TODAY



SCOPED



570 call sites unchanged · one line to roll back

Every queryset already asks the principal. We hand it a narrower one.

A token becomes a thing you can see

Legacy token

full account access · no name · unknown usage
→ replace with a scoped token

CI — nightly refresh

scope_version 2

malaria-2026

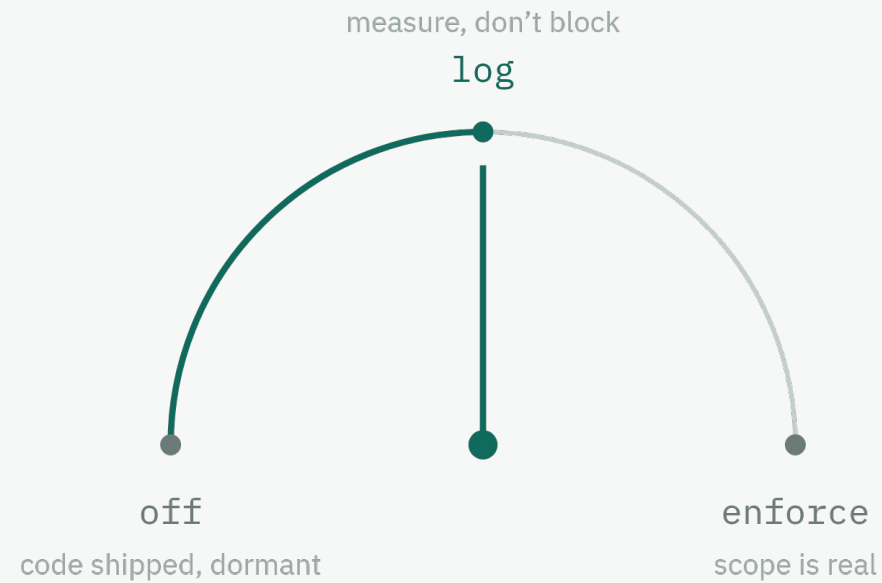
ohx_ws_7f3a...

last used 2 hours ago · 41.74.x.x

Revoke

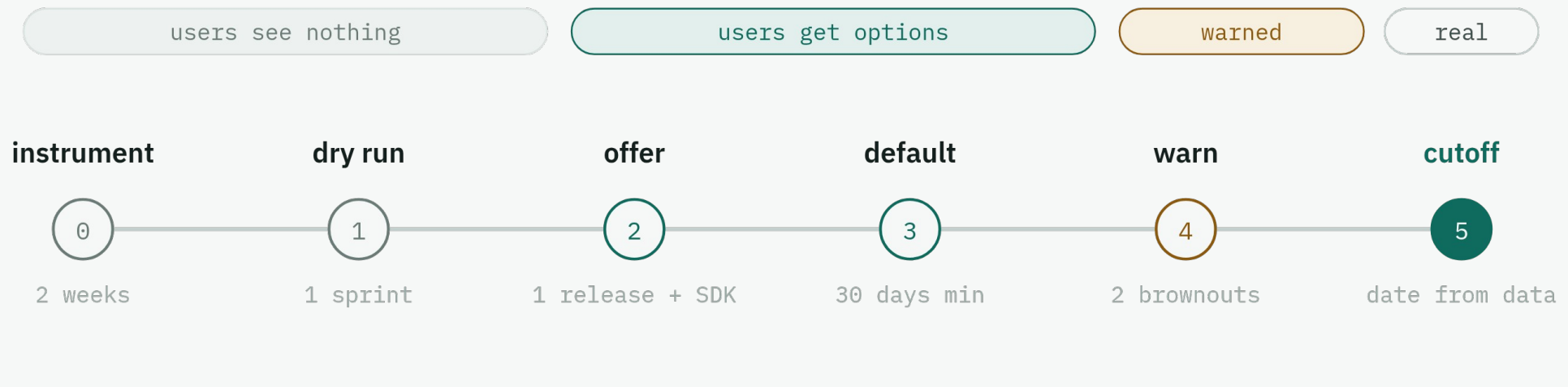
Named, per-workspace, hashed, individually revocable — and it tells you if anything still uses it.

And a switch with three positions



The plan that follows is mostly this dial, turned one notch at a time — and reversible at every notch.

Six notches, and nobody finds out from a broken pipeline



Every phase has an exit gate. Every phase is one notch back to safety.

The code ships in phase 0 – dormant. The behaviour ships last.

Durations are relative on purpose: phase 0's data is what sets the cutoff date.

First we watch. Nothing changes for anyone.

SHADOW LOG

ohx_7f3a malaria-2026 pipelines.push	ok
ohx_7f3a malaria-2026 datasets.get	ok
ohx_2b91 tb-surv workspaces.list	would deny
ohx_7f3a malaria-2026 files.upload	ok
...	

WHAT WE LEARN

which tokens are alive
who ever crosses the ring

USER IMPACT

none

Dial at `log`. Requests still succeed. We are buying facts, not shipping a change.

Two weeks of this replaces every guess about how much breakage there is.

Then we offer — and only later make it the default

Tokens

New token

(unnamed)

legacy · full account access

last used 3 days ago

Replace →

CI — nightly refresh

scoped · malaria-2026

last used 2 hours ago

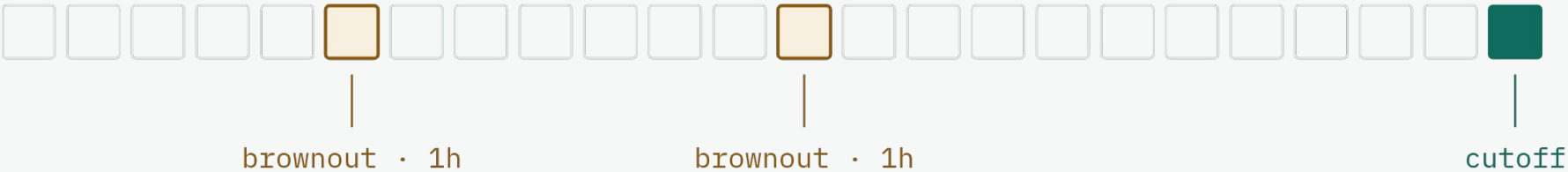
Revoke

Phase 3 adds: banner only if the legacy token was used recently · org-admin report

Creating a scoped token never invalidates the old one. Migration happens one integration at a time.

Then we warn, twice, with the lights on

THE RUN-UP



EMAIL, BEFORE EACH

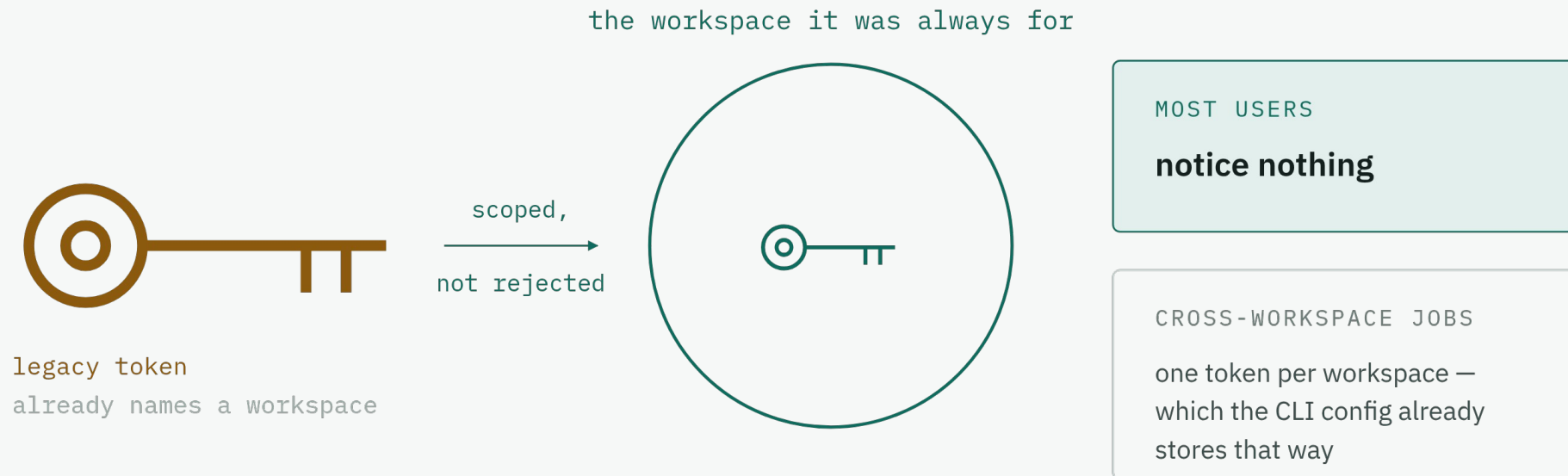
only holders of legacy tokens still in use
token name · workspace · last used · date

WHY A BROWNOUT

An hour of enforcement, announced,
beats a surprise outage on cutoff day.

Windows are picked against the real pipeline schedule — not against our timezone.

At cutoff, old tokens get *scoped* — not revoked



Verified against the SDK, not assumed. Plus a one-shot grace extension per org, with a hard expiry.

Seven ways to find out. We want the top of the list.

BEST

- They read the changelog
- The token list marks theirs **legacy**
- A banner — because it was used recently
- Their org admin tells them
- An email naming the token and the date
- A brownout fails a run — with a named error
- **Cutoff day** — nobody should reach here

WORST

Every rung above the last one is cheaper than the support ticket it prevents.

What we need to decide

01

The cutoff date

after phase 0 gives us
two weeks of numbers

02

A CLI release?

nicer scope errors, a
legacy-token warning —
optional, but before phase 4

03

Who tells users

owner for banners,
emails, docs, changelog

NOT NOW

Workspace bot identities · capability-scoped tokens (read-only, pipeline-only) · token expiry policy

Checked against the SDK: already one token per workspace, and nothing enumerates workspaces.

Three layers, three different questions

LAYER	QUESTION	MECHANISM
visibility	What rows exist for me?	<code>WorkspaceQuerySet._scoped()</code> , applied after every branch — so it also clamps the <code>self.all()</code> superuser shortcuts. Cascades to every model whose <code>filter_for_user</code> funnels through <code>Workspace.objects.filter_for_user</code> .
authority	May I act on this row?	<code>Workspace.get_membership()</code> returns <code>None</code> off-scope, so every permission built on <code>has_role()</code> collapses to <code>False</code> — about thirty checks, one method.
guard	Did something reach an object out of band?	<code>WorkspaceTokenUser.has_perm()</code> refuses objects outside the scope. Redundant on purpose — it catches the check somebody writes by hand next quarter.

The `has_role()` refactor is what makes this possible: it replaced ~30 hand-rolled membership queries with one method. Without it, scoping would need thirty correct edits and would regress on the thirty-first.

Reach, not rows — the full map

In scope, even when the row belongs elsewhere

- **Datasets shared organisation-wide.** `filter_for_user` admits `shared_with_organization` datasets from any workspace in an org you belong to.
- **Linked datasets.** `DatasetLink` is the cross-workspace mechanism — scoping must respect links, not undo them.
- **The organisation**, as the boundary those shares are evaluated against — but only the one the scoped workspace belongs to.
- **Pipeline templates** from other workspaces, as far as a member of the scoped workspace already sees them.
- **The SDK has a caller for this:** `get_dataset(..., source_workspace_slug=...)` resolves `datasetLinkBySlug` against the *source* workspace.

Out of scope, always

- Other workspaces and their memberships, connections, pipelines, webapps, saved queries.
- Every account-level mutation — profile, 2FA, org membership, creating workspaces, accepting invitations.
- The superuser shortcut. A superuser's token is no wider than anyone else's.

`Organization.filter_for_user` narrows through the relation (`workspaces__id`), not by dereferencing the workspace — so org-shared reach survives while org administration doesn't.

Why the principal, and not request.workspace

The authorization API has two shapes, and neither takes a request:

```
Model.objects.filter_for_user(user) 210 sites
user.has_perm(perm, obj)             363 sites
```

`has_perm` routes through Django's auth backend, whose signature is fixed at `(user_obj, perm, obj)`. The request is not in scope there and never will be.

So a request-based scope means either threading an argument through ~570 call sites, or ambient thread-local state the model layer reads implicitly.

And it wouldn't cover everything. `filter_for_user` is also called with no request at all — `query_runner.py`, `workspace_copier`, `core/search.py`, the connector credential modules. A request-based scope is silently *absent* there, and absent means unscoped.

What we keep `request.workspace` for: telemetry. It is the natural key for the shadow log and the scope-error metric. It just isn't the enforcement mechanism.

The cost: `request.user` is no longer plainly a `User`. Proxy identity shows up in audit logs, and per-user caches need the scope in their key.

The token record

WorkspaceAccessToken

```

workspace  FK → Workspace  the scope, immutable
user       FK → User       the identity it acts as
name       str             "CI — nightly refresh"
prefix     str indexed     shown in UI, greppable
token_hash str             sha256, shown once
scope_version int          1 legacy · 2 scoped
expires_at dt null
last_used_at dt null      throttled write
last_used_ip str null
revoked_at dt null
revoked_by FK → User null

```

Why not keep `membership.access_token`: one token per member per workspace, no name, no usage record, and revocation that breaks every copy. It can't carry the migration, the UI, or the measurement.

Hash, don't store. Today the secret is recoverable from the database. Hashing means a DB dump is not a token dump.

Prefix the string (`ohx_ws_...`) so secret scanning spots leaks, and so support can identify a token from a log line without ever seeing it.

scope_version on the record, not inferred from payload shape, is what makes the rollout measurable and reversible per token.

The error is part of the migration

Out of scope today gives `WORKSPACE_NOT_FOUND`, an empty list, or a bare `PERMISSION_DENIED`. To a user that reads as “the platform lost my workspace” — and becomes a ticket. What we return instead:

TOKEN_SCOPE_EXCEEDED

This token is scoped to workspace malaria-2026 and cannot access tb-surveillance. Create a token for that workspace in Account settings → Tokens.

The SDK and CLI surface it verbatim and link the migration page. Every occurrence is logged with the token prefix — which is what lets the org-admin report say “three of your tokens hit scope errors this week”.

And it must not be `UNAUTHENTICATED`. The CLI collapses that code into `InvalidTokenError` (`cli/api.py:213-215`) — the user would be told their token is invalid and would regenerate it. The wrong fix, applied confidently.

On disclosure: the holder already knows their own workspace list, so we reveal nothing they couldn’t learn by logging in.

Risk register

RISK	LIKELIHOOD	IMPACT	MITIGATION · DETECTION
Org-shared or linked datasets cut off by over-eager scoping	HIGH	HIGH	“Reach, not rows” semantics · tests assert presence and absence
Cross-workspace automation breaks at cutoff	MEDIUM	HIGH	Phase 0 names who · brownouts · shadow logs
A permission check bypasses <code>has_role()</code>	MEDIUM	HIGH	Guard layer catches it · test forbidding role queries outside the choke point
A new model is added later without scoping	OVER TIME	MEDIUM	SCOPED_MODELS guard test in CI
MCP server enumerates workspaces	MEDIUM	MEDIUM	Audit its operations before phase 3 · shadow logs by user agent
Ops scripts using superuser tokens break	MEDIUM	MEDIUM	Intentional — give ops an audited path, not a wide token
Rollout stalls at “legacy still in use” forever	MEDIUM	MEDIUM	Announced dates · brownouts · one-shot grace extension

Alternatives, and why not

OPTION	WHY NOT
Per-request workspace header — token stays wide, client declares intent	Client-declared scope isn't a security boundary; a leaked token just omits the header
JWT with claims instead of signed opaque payloads	Real benefits, bigger change, and no revocation without a denylist. The signed-payload strategy is adequate once the token table gives us revocation
Capability tokens now (read-only, pipeline-only)	Doubles the permission surface while the primary hole is still open. Ship workspace scoping; keep the extension point
Enforce in each resolver	~200 resolvers, every one a chance to forget, unreviewable, and no single rollback switch
Hard cut in one release	Fastest to a clean codebase, and the one option guaranteed to break users without warning
Do nothing, document the risk	The token is an account credential in a place users treat as a project credential. That gap doesn't stay theoretical

Where the branch stands

Built

- `WorkspaceScopedPrincipal` marker, with `ServicePrincipal` inheriting from it
- `WorkspaceTokenUser` — the `User` proxy narrowed to one workspace
- `Workspace.has_role()`, replacing ~30 hand-rolled membership queries
- `_scoped()` post-filter on the workspace queryset, including superuser shortcuts
- `test_visibility_scoping.py` — the matrix, run for a member and a superuser holder

Left to do

- Wire the middleware (one line) behind the three-state flag
- Scope as a constructor argument, not a mutated class attribute
- Ask the principal, not `isinstance`, from `core/models/base.py`
- Registry dispatch on a payload `version`, replacing the `if/elif`
- Guard test so a new model can't skip `SCOPED_MODELS`
- Decide `bypass_two_factor` for tokens
- Tests for org-shared and linked datasets — the likeliest quiet break

How a brownout actually works

Why do it

- Emails get ignored. A failed run at 10:00 on an announced Tuesday does not.
- Failure is *recoverable* — a nightly pipeline can be re-run the same day. The identical failure on cutoff day is an outage with no fallback.
- It rehearses the enforce path in production before it is permanent.
- It turns “we think a few tokens still cross workspaces” into a list of names.

Blast radius is bounded by construction: only token-authenticated requests are affected. Everyone in the web UI sees nothing.

What it costs to build: nearly nothing. A brownout is a schedule, not a feature — and the lever exists: [Feature / FeatureFlag](#) ([user_management/models.py:840-865](#)), with [force_activate](#) as a global switch in Django admin. No deploy, no restart.

- [has_feature_flag](#) is an uncached query per call — memoise it before the auth path.
- A boolean can’t express [off / log / enforce](#): use a mode plus an [enforce_from/until](#) window, so brownouts run themselves.
- No scheduler in [settings/base.py](#) — a timed flip means a CronJob, or a person.

The hard parts aren’t code: the hour (real pipeline schedules, many timezones), the comms, no auto-retry, and someone watching the metric.

What the CLI and SDK actually do

FINDING	WHERE
Config is already one token per workspace — <code>[workspaces]</code> maps slug → token, and <code>access_token</code> returns the current workspace's	cli/settings.py:82,117
<code>workspaces list / activate / rm</code> are local only — no API call	cli/cli.py:141-160
<code>workspaces add</code> validates with <code>getWorkspace(slug)</code> — singular, the token's own workspace	cli/api.py:225
Nothing calls <code>me</code> , the plural <code>workspaces</code> , or <code>organizations</code> — present in the generated client, unused	grep of cli/ + sdk/
Every runtime call names its workspace explicitly	sdk/workspaces/ current_workspace.py
User-Agent is versioned — Phase 0 measures the client version spread for free	cli/api.py:194
No token-format validation anywhere — <code>ohx_ws_...</code> works on every installed client	cli/api.py:193

The one exposure: `get_dataset(..., source_workspace_slug=...)` is an explicit cross-workspace read. It survives under “reach, not rows” — but the scoping fixture only builds a *private* link in the other workspace, so nothing protects it yet.